

ui.slide_item 全面详解

`ui.slide_item` 是 NiceGUI 基于 Quasar 的 `QSlideItem` 组件封装的滑动交互元素，自版本 2.12.0 起新增，支持向上下左右四个方向滑动触发自定义操作，兼具灵活性与易用性，适用于列表项滑动操作、卡片交互等场景。

核心特性与基础概念

核心功能

- 多方向滑动支持：可配置左、右、上、下四个方向的滑动操作区
- 自定义内容与样式：支持嵌套普通文本或复杂 UI 元素，滑动区背景色可灵活配置
- 事件驱动：滑动触发回调函数，支持全局与方向专属事件处理
- 状态重置：滑动操作后可通过 API 恢复初始状态
- 插槽机制：通过插槽（Slot）实现滑动区与主内容区的灵活布局

依赖与兼容性

- 基于 Quasar 组件库，支持 Quasar/Tailwind/CSS 颜色规范
- 要求 NiceGUI 版本 $\geq 2.12.0$ （部分属性如 `html_id` 需 $\geq 2.16.0$ ）
- 支持与 `ui.list` 等容器组件搭配使用，适配响应式布局

初始化参数

参数名	类型	说明	默认值
text	str	显示文本，传入后会自动创建嵌套的 <code>ui.item</code> 元素	""
on_slide	Handler[SlideEventArguments]	所有滑动操作触发时的全局回调函数	None

示例：基础初始化

```
from nicegui import ui

with ui.list().props('bordered separator'):
    # 带默认文本的滑动项
    with ui.slide_item('向左或向右滑动我', on_slide=lambda e: ui.notify(f'滑动方向: {e.side}')) as slide1:
        slide1.left('左侧操作', color='green')
    # 空文本滑动项（后续可自定义内容）
    with ui.slide_item() as slide2:
        ui.item('自定义内容')

ui.run()
```

关键属性

属性名	类型	说明	新增版本
classes	Classes[Self]	元素的 CSS 类名，用于自定义样式	-
client	Client	元素所属的客户端实例	-
enabled	BindableProperty	是否启用元素（支持双向绑定）	-
html_id	str	HTML DOM 中的元素 ID	2.16.0
is_deleted	bool	元素是否已被删除	-
is_ignoring_events	bool	是否忽略事件触发	-
parent_slot	Slot None	父插槽引用（可修改）	-
props	Props[Self]	组件属性（继承自 Quasar 组件）	-
style	Style[Self]	内联 CSS 样式	-
visible	BindableProperty	元素可见性（支持双向绑定）	-

属性使用示例

```
# 绑定可见性：当变量为 True 时显示
show_slide = True
with ui.slide_item('绑定可见性') as slide:
    slide.bind_visibility(target_object=locals(), target_name='show_slide')
ui.checkbox('显示滑动项', value=show_slide, on_change=lambda e: show_slide(e.value))
```

核心方法

1. 滑动操作区配置方法

用于在指定方向添加滑动触发的操作区，支持快捷方法与通用方法：

快捷方法（推荐）

方法名	说明	参数与 action 方法一致
left()	左侧滑动操作区	-
right()	右侧滑动操作区	-
top()	顶部滑动操作区	-
bottom()	底部滑动操作区	-

通用方法：action ()

```
def action(  
    side: SlideSide, # 滑动方向: "left"/"right"/"top"/"bottom"  
    text: str = '', # 操作区显示文本  
    on_slide: Handler[SlideEventArguments] | None = None, # 该方向滑动的专属回调  
    color: str | None = 'primary' # 背景色 (支持 Quasar/Tailwind/CSS 颜色)  
) -> Slot
```

示例：多方向操作区配置

```
from nicegui import ui  
  
with ui.list().props('bordered'):  
    with ui.slide_item('上下左右均可滑动') as slide:  
        # 左侧: 绿色背景, 点击后重置  
        with slide.left('呼叫', color='green', on_slide=lambda: ui.notify('正在呼叫')):  
            ui.icon('phone')  
        # 右侧: 红色背景  
        slide.right('删除', color='red', on_slide=lambda: ui.notify('删除成功'))  
        # 顶部: 蓝色背景  
        slide.top('置顶', color='blue')  
        # 底部: 紫色背景 (使用通用 action 方法)  
        slide.action('bottom', '分享', color='purple')  
  
ui.run()
```

2. 状态管理方法

方法名	说明	参数
reset()	重置滑动项到初始状态 (核心方法)	-
enable()	启用元素	-
disable()	禁用元素	-
set_enabled(value: bool)	设置启用状态	value: True = 启用, False = 禁用
set_visibility(visible: bool)	设置可见性	visible: True = 显示, False = 隐藏

重置功能示例

```
from nicegui import ui

with ui.list().props('bordered'):
    with ui.slide_item() as slide:
        ui.item('滑动后可重置')
        slide.left('左侧操作', color='blue')
        slide.right('右侧操作', color='purple')
# 外部按钮触发重置
ui.button('重置滑动项', on_click=slide.reset)

ui.run()
```

3. 事件处理方法

方法名	说明	参数
on_slide(side: SlideSide None, handler: Handler[SlideEventArguments])	绑定滑动事件，指定方向或全局触发	side: 指定方向（None 表示全局）， handler: 回调函数
on(type: str, handler: Handler, ...)	通用事件绑定（支持 click、mousedown 等）	详见事件绑定章节

事件处理示例

```
from nicegui import ui

with ui.list().props('bordered'):
    with ui.slide_item('滑动事件演示', on_slide=lambda e: ui.notify(f'全局事件: {e.side}')) as slide:
        # 左侧专属事件（优先级高于全局）
        slide.left('A', on_slide=lambda e: ui.notify(f'左侧事件: {e.side}'))
        # 右侧专属事件
        slide.right('B', on_slide=lambda e: ui.notify(f'右侧事件: {e.side}'))
        # 绑定点击事件
        slide.on('click', lambda: ui.notify('点击了滑动项'))

ui.run()
```

4. 布局与内容管理方法

方法名	说明	关键参数
add_slot(name: str, template: str None = None)	添加插槽（适配 Vue 插槽机制）	name: 插槽名称, template: Vue 模板
clear()	移除所有子元素	-
delete()	删除元素及其所有子元素	-

方法名	说明	关键参数
move(target_container: Element None = None, ...)	移动元素到其他容器	target_container: 目标容器, target_index: 位置索引
remove(element: Element int)	移除指定子元素	element: 元素实例或 ID

5. 绑定方法（双向绑定支持）

方法名	说明	绑定方向
bind_enabled(...)	绑定启用状态	双向
bind_enabled_from(...)	从目标对象绑定启用状态	单向（目标→元素）
bind_enabled_to(...)	向目标对象绑定启用状态	单向（元素→目标）
bind_visibility(...)	绑定可见性	双向
bind_visibility_from(...)	从目标对象绑定可见性	单向（目标→元素）
bind_visibility_to(...)	向目标对象绑定可见性	单向（元素→目标）

绑定示例：启用状态与输入框联动

```
from nicegui import ui

enable_slide = ui.checkbox('启用滑动操作', value=True)
with ui.slide_item('联动启用状态') as slide:
    slide.left('编辑', color='orange')
    # 绑定启用状态到复选框
    slide.bind_enabled_from(target_object=enable_slide, target_name='value')

ui.run()
```

6. 其他常用方法

方法名	说明	关键参数
add_dynamic_resource(name: str, function: Callable)	添加动态资源	function: 资源生成函数
add_resource(path: str Path)	添加静态资源 (CSS/JS 等)	path: 资源路径
ancestors(include_self: bool = False)	迭代祖先元素	include_self: 是否包含自身
descendants(include_self: bool = False)	迭代后代元素	include_self: 是否包含自身
default_classes(...)	批量设置默认 CSS 类	add/remove/toggle/replace: 类名操作

方法名	说明	关键参数
default_props(...)	批量设置默认属性	add/remove: 属性操作
default_style(...)	批量设置默认样式	add/remove/replace: 样式操作
get_computed_prop(prop_name: str, timeout: float = 1)	获取计算属性 (需等待)	prop_name: 属性名
mark(*markers: str)	标记元素 (用于测试查询)	markers: 标记字符串
run_method(name: str, *args: Any, ...)	运行客户端方法	name: 方法名, args: 参数
tooltip(text: str)	添加 tooltip 提示	text: 提示文本
update()	刷新客户端元素状态	-

高级用法示例

1. 复杂布局与自定义 UI

```
from nicegui import ui

with ui.list().props('bordered'):
    with ui.slide_item() as slide:
        # 主内容区: 带头像和多级文本的复杂布局
        with ui.item():
            with ui.item_section().props('avatar'):
                ui.icon('person', size='24px')
            with ui.item_section():
                ui.item_label('Alice A. Anderson', props='font-weight-bold')
                ui.item_label('CEO').props('caption text-gray-500')
        # 左侧滑动区: 呼叫功能 (点击后重置)
        with slide.left(on_slide=lambda: ui.notify('正在呼叫 Alice...')):
            with ui.item(on_click=slide.reset):
                with ui.item_section().props('avatar'):
                    ui.icon('phone', color='white')
                ui.item_section('呼叫', props='text-white')
        # 右侧滑动区: 发送消息
        with slide.right(on_slide=lambda: ui.notify('发送消息给 Alice...')):
            with ui.item(on_click=slide.reset):
                ui.item_section('消息', props='text-white')
                with ui.item_section().props('avatar'):
                    ui.icon('message', color='white')

ui.run()
```

2. 滑动事件精细化控制

```
from nicegui import ui

with ui.list().props('bordered'):
    with ui.slide_item('滑动事件演示') as slide:
        # 全局滑动事件
        slide.on_slide(None, lambda e: ui.notify(f'全局监听: {e.side} 方向滑动'))
        # 左侧专属事件
        slide.left('同意', color='green', on_slide=lambda e: ui.notify(f'同意操作 ({e.side})'))
        # 右侧专属事件
        slide.right('拒绝', color='red', on_slide=lambda e: ui.notify(f'拒绝操作 ({e.side})'))

ui.run()
```

3. 批量管理多个滑动项

```
from nicegui import ui

slide_items = [] # 存储滑动项实例用于批量操作

with ui.list().props('bordered'):
    for i in range(3):
        with ui.slide_item(f'滑动项 {i+1}') as slide:
            slide.left('编辑', color='blue')
            slide.right('删除', color='red')
            slide_items.append(slide)

# 批量重置按钮
ui.button('重置所有滑动项', on_click=lambda: [slide.reset() for slide in slide_items])
# 批量禁用按钮
ui.button('禁用所有滑动项', on_click=lambda: [slide.disable() for slide in slide_items])

ui.run()
```

注意事项与最佳实践

- 容器搭配**: 建议与 `ui.list` 组件搭配使用, 通过 `bordered` `separator` 等属性优化视觉效果
- 颜色规范**: `color` 参数支持 Quasar 预设颜色 (如 `primary` `secondary`)、Tailwind 颜色 (如 `bg-blue-500`) 或 CSS 颜色值 (如 `#ff0000`)
- 性能优化**: 复杂布局下避免过度嵌套元素, 批量操作时使用列表存储实例而非重复查询
- 事件优先级**: 方向专属 `on_slide` 回调优先级高于全局 `on_slide`, 触发时会同时执行
- 版本兼容**: 使用 `html_id` 等属性时需确认 NiceGUI 版本, 避免兼容性问题
- 重置时机**: 建议在滑动操作完成后 (如按钮点击) 调用 `reset()` 方法, 提升用户体验

常见问题排查

- **滑动无响应**: 检查是否嵌套在正确的容器中 (如 `ui.list`) , 确认 `enabled` 属性为 `True`
- **样式异常**: 通过 `classes` 或 `style` 覆盖默认样式, 避免与 Quasar 预设样式冲突
- **事件不触发**: 确认回调函数参数正确 (需接收 `slideEventArguments` 实例) , 检查 `is_ignoring_events` 状态
- **重置无效**: 确保调用 `reset()` 方法的对象是 `ui.slide_item` 实例, 而非子元素